UNIVERSITÀ DELLA CALABRIA

DIPARTIMENTO DI INGEGNERIA INFORMATICA, MODELLISTICA, ELETTRONICA E SISTEMISTICA

CORSO DI LAUREA TRIENNALE IN INGEGNERIA INFORMATICA

ELETTRONICA DIGITALE

RELAZIONE TERZO PROGETTINO – PIPELINE ADDER

Petrassi Maria Jose' – 251439

Ponte Andrea – 251050

Ruffolo Emilio – 250521

Traccia Del Progetto

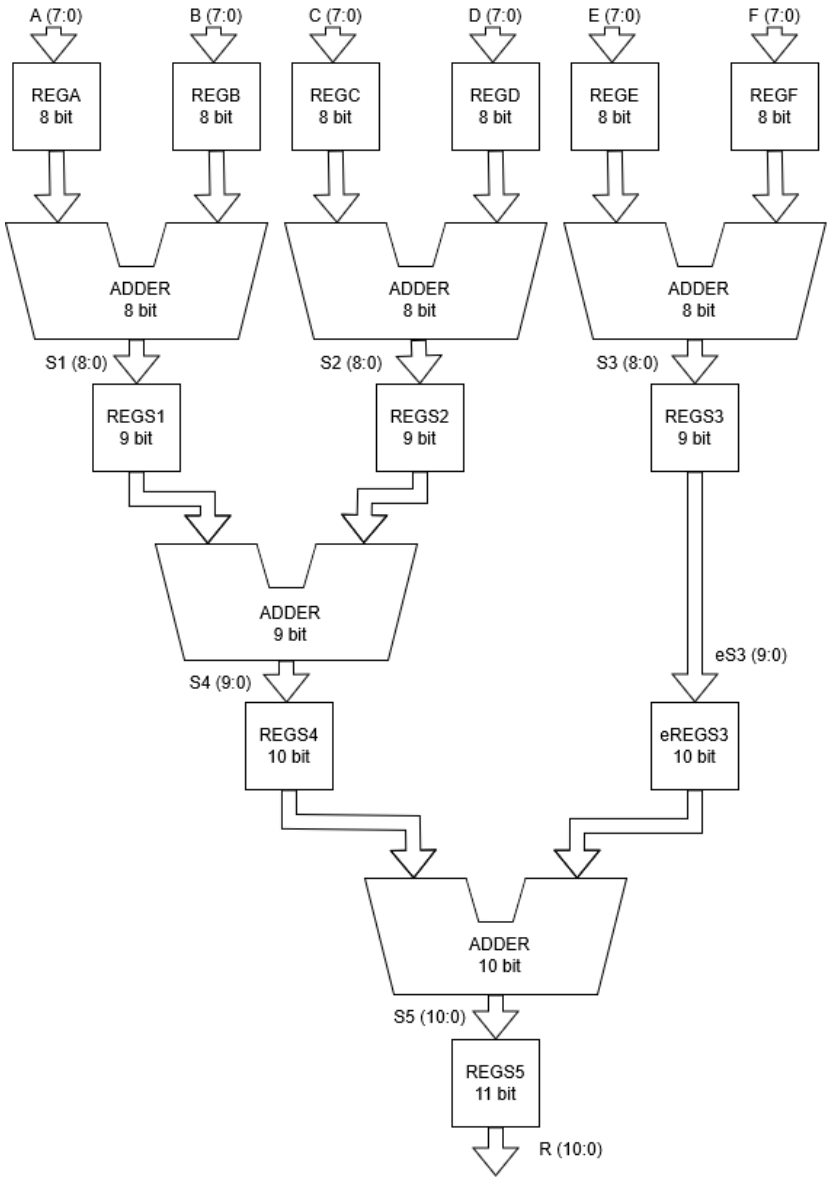
Si richiede la progettazione di un circuito sommatore in grado di elaborare 6 operandi a 8 bit, rappresentati in complemento a due. L'architettura dovrà presentare una struttura ad albero con pipeline integrale: ogni livello della gerarchia di addizione deve essere seguito da uno stadio di registri di sincronizzazione.

La validazione del sistema deve essere effettuata tramite:

1. Simulazione behavioural;
2. Simulazione post-synthesis;
3. Simulazione post-implementation.

Infine, si richiede la caratterizzazione hardware del dispositivo, riportando i seguenti parametri:

1. Utilizzo delle risorse: numero di LUT (Look-Up Tables) e Flip-Flop impiegati;
2. Analisi energetica: stima della potenza totale dissipata;
3. Prestazioni temporali: valutazione della massima frequenza di clock operativa.



1. Architettura del Circuito

Il circuito progettato è un sommatore a 6 operandi con architettura pipelined ad albero. Questa struttura è stata scelta per massimizzare il throughput e la frequenza operativa, permettendo al sistema di processare nuovi dati a ogni colpo di clock dopo una latenza iniziale.

1.1 Struttura ad albero e gestione del bit di riporto

L'architettura segue una topologia ad albero per ridurre il cammino critico complessivo rispetto ad una struttura a cascata. Andando a sommare sei operandi a $8\ bit$ in complemento a due, il numero di bit necessari per rappresentare il risultato finale senza overflow è calcolato come:

$$n_{out} = n_{in} + \left[\log_2\left(N_{operandi}\right)\right] = 8 + [\log_2(6)] = 8 + 3 = 11\ bit$$

Il progetto gestisce l'espansione del range dinamico a ogni livello della gerarchia:

- **Livello 1**: Tre addizioni parallele ($A + B, C + D, E + F$) che portando il dato da 8 a 9 bit;
- **Livello 2**: Addizione dei primi due risultati parziali ($9 + 9\ bit \rightarrow 10\ bit$) e bilanciamento del terzo ramo;
- **Livello 3**: Addizione finale ($10 + 10\ bit \rightarrow 11\ bit$).

1.2 Pipeline integrale e bilanciamento

Per garantire la sincronizzazione dei segnali, l'architettura è integralmente pipelined. Ogni operazione logica è separata da uno stadio di memorizzazione, ovvero dei registri:

- **Registri di ingresso**: Vanno da REGA – REGF, campionano gli operandi in ingresso per stabilizzare i dati prima dell'elaborazione;
- **Registri intermedi**: Ogni componente adder nel codice include un processo sincrono che registra l'uscita. Questo riduce il ritardo di propagazione tra i livelli, limitandolo al solo ritardo della logica combinatoria di un singolo sommatore.
- **Bilanciamento del ramo**: Avviene ad $E + F$. Poiché la somma di E ed F viene completata mentre le altre quattro variabili stanno ancora venendo sommate tra loro, è stato inserito un registro di bilanciamento (RegSum3) e una successiva estensione del segno (eRS3) per garantire che i dati arrivino al sommatore finale nello stesso ciclo di clock.

1.3 Componenti VHDL

Il design è di tipo gerarchico – strutturale, basato sul riuso di due componenti principali definiti tramite generics:

- **registro**: Un registro parallelo di dimensione n parametrizzabile, fondamentale per la sincronizzazione;
- **adder**: Un sommatore che implementa la logica di addizione e l'estensione del segno. Dall'analisi del codice, il sommatore calcola internamente i segnali di propagate e generate e registra il risultato finale su un fronte di salita del clock.

1.4 Analisi della latenza

Data la struttura del codice e del diagramma, la latenza totale (espressa in cicli di clock T_{ck}) dal momento in cui l'input è stabile a quando il risultato R è disponibile in uscita è pari a 4 cicli di clock:

- **Input Registering**: REGA – F;
- **Primo stadio di somme**: S1, S2, S3;
- **Secondo stadio**: S4 e bilanciamento S3;
- **Terzo stadio e output finale**: S5.

2. Implementazione del circuito in VIVADO

Il progetto è realizzato in VHDL mediante un approccio modulare gerarchico. Si compone di due unità di base (Registro e adder) istanziate all'interno dell'unità top-level, pipe_adder, per formare l'albero di somma a tre stadi.

Per garantire la scalabilità del data-path e la gestione della crescita di bit (causata dal sommatore) attraverso i livelli della pipeline, entrambi i componenti base sono definiti tramite il generico n.

2.1 Registro

Implementa un banco a n bit di Flip-Flop con aggiornamento sincrono sul fronte di salita del clock. È utilizzato sia per il campionamento degli ingressi che per il bilanciamento temporale dei rami dell'albero.

```
entity Registro is
    generic (n: integer);
    Port ( D : in STD_LOGIC_VECTOR (n-1 downto 0);
        clk : in STD_LOGIC;
        Q : out STD_LOGIC_VECTOR (n-1 downto 0));
end Registro;

architecture MyRegistro of Registro is
    begin

    process(clk) begin
        if rising_edge(clk) then
            Q<=D;
        end if;
    end process;

end MyRegistro;
```

L'entity è definita con un'architettura parametrica basata sul generico n che determina la larghezza dei vettori Input (D) e Output(Q). È poi presente il segnale di sincronismo (clk) rappresentante il segnale di clock globale del sistema.
L'architecture è resa sensibile esclusivamente al segnale clk e descrive la logica di trasferimento dati tipica di un registro a flip flop sensibile ai fronti in salita.
Grazie alla parametrizzazione, una singola descrizione VHDL viene utilizzata per istanziare registri di ampiezza variabile necessari. In particolare a 8 bit per il campionamento degli ingressi operandi (A,…,F), 9 e 10 bit per la memorizzazione dei risultati parziali, e 11 bit per la memorizzazione della somma finale.

2.2 Adder

Il modulo adder rappresenta l'unità di elaborazione fondamentale della pipeline. A differenza di un classico sommatore combinatorio, questa entità incapsula sia la logica aritmetica che il registro di pipeline di uscita in un singolo ciclo di clock.

```
entity adder is
    generic(n : integer := 8);
    port(
        A,B : in std_logic_vector(n-1 downto 0);
        clk : in STD_LOGIC;
        S : out std_logic_vector(n downto 0)
    );
end adder;

architecture MyAdder of adder is

component registro is
    generic (n: integer);
    Port ( D : in STD_LOGIC_VECTOR (n-1 downto 0);
        clk : in STD_LOGIC;
        Q : out STD_LOGIC_VECTOR (n-1 downto 0));
end component;

    signal C: std_logic_vector (n+1 downto 0);
    signal p, g: std_logic_vector (n downto 0);
    signal R: std_logic_vector (n downto 0);

    begin
        C(0) <= '0';

        p <= (A(n-1) xor B(n-1)) & (A xor B);
        g <= (A(n-1) and B(n-1)) & (A and B);
        C(n+1 downto 1) <= g or (p and C(n downto 0));

        process(clk) begin
            if rising_edge(clk) then
                S <= p xor C(n downto 0);
             end if;
        end process;

end MyAdder;
```

L'entity è anche questa volta parametrizzata tramite il generico n per adattarsi ai diversi stadi dell'albero di somma. Gli ingressi (A e B) sono vettori a n bit in complemento a due, mentre l'uscita (S) è un vettore a n+1 bit . Questa differenza dimensionale è necessaria per far sì che l'output includa nativamente il bit di riporto/segno generato, prevenendo l'overflow locale.
Il calcolo della somma è descritto strutturalmente seguendo la logica tipica del Ripple Carry Adder tramite i segnali vettoriali propagate e generate, e tramite la catena dei riporti C.
Al fronte di salita del clock, il risultato della somma viene calcolato come $S = p \oplus C$ e memorizzato direttamente nel registro di uscita. Questa operazione introduce un registro implicito che isola temporalmente gli stadi successivi della pipeline.

2.3 Pipe Adder

Questo rappresenta il vertice della gerarchia di progetto. Implementa una rete di calcolo sincrona ad albero per la somma di 6 operandi, strutturata su più stadi per massimizzare la frequenza di elaborazione.

```
4   entity pipe_adder is
5       Port (
6           A, B, C, D, E, F : in std_logic_vector(7 downto 0);
7           clk: in std_logic;
8           R: out std_logic_vector(10 downto 0)
9       );
10  end pipe_adder;
13  architecture MyPipeAdder of pipe_adder is
14
15  component registro is...
21
22  component adder is...
30
31
32  signal RA, RB, RC, RD, RE, RF: std_logic_vector (7 downto 0);
33  signal RS3: std_logic_vector (8 downto 0);
34  signal eRS3: std_logic_vector (9 downto 0);
35
36  signal S1, S2, S3: std_logic_vector (8 downto 0);
37  signal S4: std_logic_vector (9 downto 0);
38  signal S5: std_logic_vector (10 downto 0);
39
40  begin
41      RegA: registro generic map (8) port map (A, clk, RA);
42      RegB: registro generic map (8) port map (B, clk, RB);
43      RegC: registro generic map (8) port map (C, clk, RC);
44      RegD: registro generic map (8) port map (D, clk, RD);
45      RegE: registro generic map (8) port map (E, clk, RE);
46      RegF: registro generic map (8) port map (F, clk, RF);
47
48      Sum1: adder generic map (8) port map (RA, RB, clk, S1);
49      Sum2: adder generic map (8) port map (RC, RD, clk, S2);
50      Sum3: adder generic map (8) port map (RE, RF, clk, S3);
51
52      RegSum3: registro generic map (9) port map (S3, clk, RS3);
53      eRS3 <= RS3(8) & RS3;
54
55      Sum4: adder generic map (9) port map (S1, S2, clk, S4);
56      Sum5: adder generic map (10) port map (S4, eRS3, clk, S5);
57
58      R<=S5;
59  end MyPipeAdder;
```

Il Data-Path seguito dai dati è strutturato in tre fasi principali:

1. **Input:** Appena arrivano i 6 numeri, questi vengono subito salvati all'interno di 6 registri (RA,…,RF), per memorizzare i dati in ingresso e renderli stabili per il calcolo successivo.
2. **Primo Livello di Somme:** I tre sommatori del primo livello lavorano parallelamente sulle coppie di numeri:
    - Il primo somma A+B e produce S1.
    - Il secondo somma C+D e produce S2.
    - Il terzo somma E+F e produce S3.

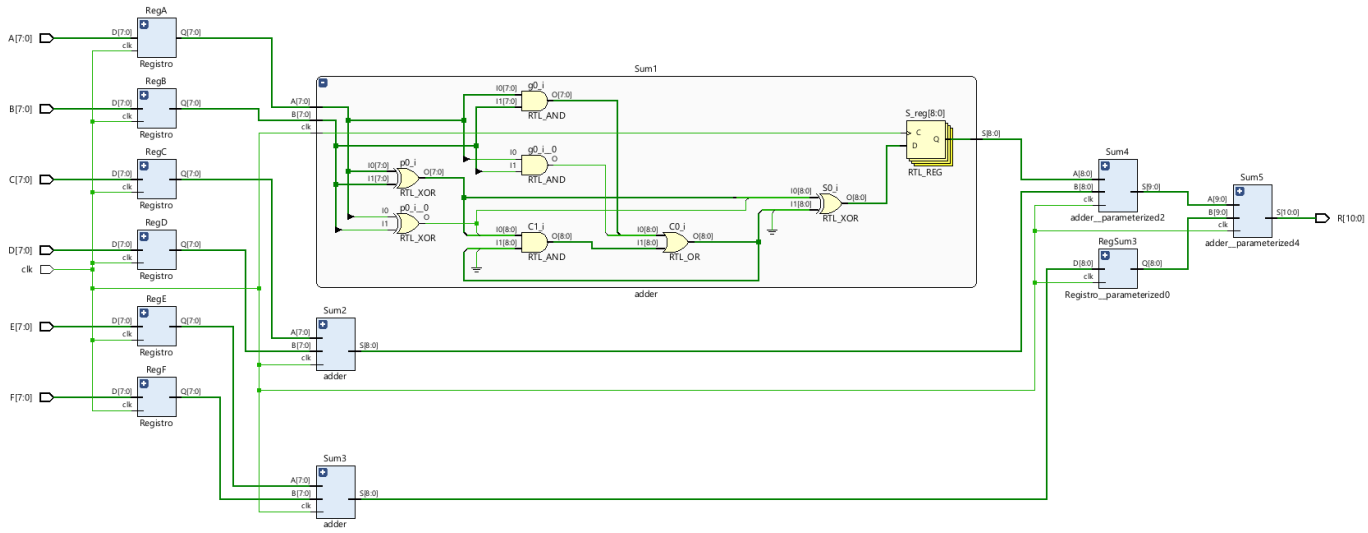
    I risultati ottenuti sono a 9 bit.

3. **Secondo Livello e Allineamento:** Il livello superiore ci ha fornito tre risultati parziali (S1, S2, S3), tuttavia abbiamo strutturato logicamente il nostro sommatore per lavorare a coppie, siamo costretti dunque a sommare i tre in due livelli distinti. In questo livello ci limitiamo a:
    - Sommare i segnali S1 e S2 tra loro dall'unità Sum4.
    - Sincronizzare il segnale S3 facendolo passare in un registro di "ritardo" (RegSum3), in modo che arrivi esattamente nello stesso istante di S4 al sommatore finale.Viene inoltre esteso di un bit in modo da raggiungere la stessa larghezza dell'altro operando.
4. **Somma Finale** L'ultimo sommatore (Sum5) riceve i due operandi e si limita a calcolare il totale finale su 11 bit, inviandolo all'uscita R.

Il funzionamento a pipeline adottato può essere considerato analogo a una catena di montaggio industriale: invece di attendere il completamento dell'intera somma prima di accettare nuovi ingressi, il sistema elabora contemporaneamente operazioni distinte in stadi differenti. Nello specifico, mentre l'ultimo livello (Sum5) calcola il risultato finale del primo set di dati, gli stadi precedenti (Sum1...Sum4) stanno già lavorando sui calcoli intermedi dei set successivi. Questa sovrapposizione temporale elimina i tempi morti ("idle"), garantendo un flusso di elaborazione continuo.

Il beneficio primario consiste nell'aumento della Frequenza Massima di Clock. In una struttura non pipelined, il segnale dovrebbe attraversare l'intera catena di sommatori in un unico ciclo, generando un bottleneck esteso che limiterebbe la velocità del sistema. Nel progetto proposto, i registri spezzano tale percorso: il segnale deve attraversare solo la logica di un singolo sommatore prima di essere memorizzato. Grazie a questo parallelismo temporale, il circuito raggiunge un Throughput ideale: a regime, il sistema è in grado di rilasciare un nuovo risultato valido ad ogni fronte di salita del clock. Il costo di tale approccio è rappresentato dalla Latenza. Dovendo attraversare l'intera profondità della pipeline (registri di ingresso più tre livelli di calcolo), il risultato corretto su R è disponibile con un ritardo fisso di quattro cicli di clock; in un contesto di elaborazione continua, tuttavia, questo ritardo iniziale risulta trascurabile.

2.4 Elaborated Design



3. Simulazione Behavioural

Per validare la correttezza funzionale e le prestazioni temporali del sommatore pipelined, è stato sviluppato un Testbench (sim_pipe_adder) che istanzia il modulo Circuit Under Test (CUT) e genera stimoli deterministici.

```
8   architecture MySimPipeAdder of sim_pipe_adder is
9
10  component pipe_adder is
11      Port (
12          A, B, C, D, E, F : in std_logic_vector(7 downto 0);
13          clk: in std_logic;
14          R: out std_logic_vector(10 downto 0)
15      );
16  end component;
17
18  signal A, B, C, D, E, F: std_logic_vector (7 downto 0);
19  signal clk: std_logic:='0';
20  signal R: std_logic_vector (10 downto 0);
21
22  constant T_clk: time:=10ns;
23
24  begin
25      CUT: pipe_adder port map (
26          A => A, B => B, C => C,
27          D => D, E => E, F => F,
28          clk => clk,
29          R => R
30      );
31      process begin
32          wait for T_clk/2;
33          clk <= not clk;
34      end process;
35      process begin
36          for i in 1 to 100 loop
37              wait until falling_edge(clk);
38              A <= conv_std_logic_vector(i, 8);
39              B <= conv_std_logic_vector(i, 8);
40              C <= conv_std_logic_vector(i, 8);
41              D <= conv_std_logic_vector(i, 8);
42              E <= conv_std_logic_vector(i, 8);
43              F <= conv_std_logic_vector(i, 8);
44          end loop;
45          for i in 1 to 5 loop
46              wait until falling_edge(clk);
47          end loop;
48          wait;
49      end process;
50
51  end MySimPipeAdder;
```

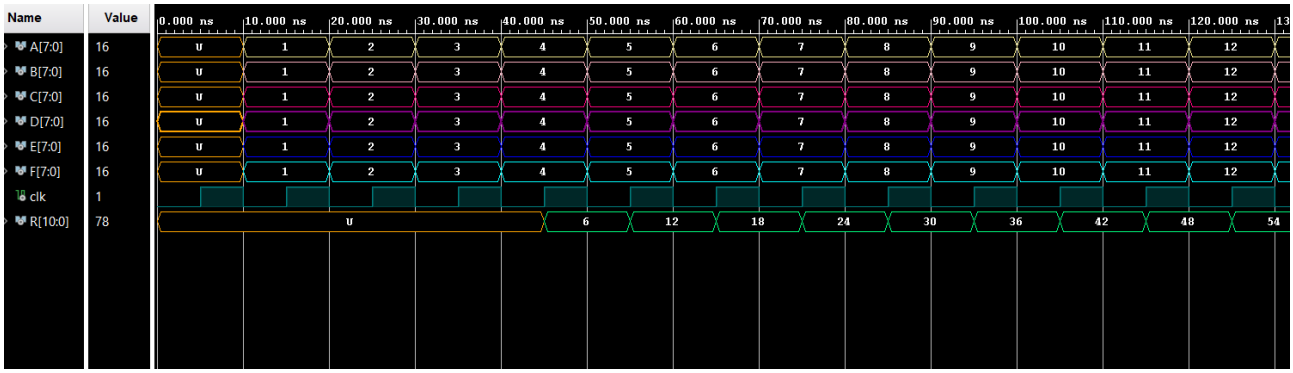
Il testbench non presenta porte di I/O verso l'esterno (entity vuota), ma definisce internamente i segnali necessari per pilotare il componente pipe_adder. La generazione del segnale di sincronismo clk è affidata a un process dedicato, che produce un'onda quadra con *duty-cycle* al 50% periodica. Il periodo è definito dalla costante T_clk (10 ns), corrispondente a una frequenza operativa di test di 100 MHz.

La verifica adotta una **generazione algoritmica degli stimoli** per valutare il comportamento del circuito sotto carico continuo, utilizzando un ciclo iterativo for che copre 100 casi di test consecutivi. All'interno del ciclo, agli ingressi viene assegnato il valore intero dell'indice convertito in vettore logico tramite la funzione conv_std_logic_vector.

È adesso fondamentale implementare una sincronizzazione degli stimoli. L'aggiornamento degli ingressi avviene sul fronte di discesa del clock (wait until falling_edge(clk)), lontano dal fronte di salita (momento in cui i registri interni campionano i dati) evitando di indurre il circuito in stato metastabile (che in alcuni casi potrebbe causare una condizione di equilibrio instabile dei transistor inducendo l'uscita del flip flop a $\frac{V_{DD}}{2}$), e garantendo che al fronte di salita successivo, i dati all'ingresso del primo stadio di pipeline siano perfettamente stabili e definiti.

Al termine della fase di iniezione degli stimoli (dopo i 100 cicli), il testbench prevede una fase di attesa. Un secondo ciclo for mantiene la simulazione attiva per ulteriori cinque periodi di clock senza introdurre nuovi dati significativi. Questa fase di Flushing è necessaria per permettere agli ultimi dati inseriti di attraversare l'intera profondità della pipeline (latenza di 4 cicli) e raggiungere l'uscita R prima dell'arresto definitivo della simulazione (wait finale).

3.1 Risultato Grafico della simulazione



Dal grafico è evidente il comportamento temporale della latenza nella pipeline. Il primo risultato valido ("6") appare esattamente in corrispondenza del quarto fronte di salita dopo l'iniezione del dato. Questo intervallo temporale conferma empiricamente la latenza strutturale teorizzata in fase di architettura. La zona arancione (U, undefined) rappresenta la fase transitoria di riempimento della pipeline.
A partire dal tempo $T = 45\,ns$, il sistema entra in regime stazionario. Si osserva che l'uscita viene aggiornata ad ogni ciclo di clock (ogni 10ns), presentando un nuovo risultato valido senza tempi di idling. Questo conferma che il circuito sostiene il Throughput massimo teorico di 1 operazione per ciclo di clock.

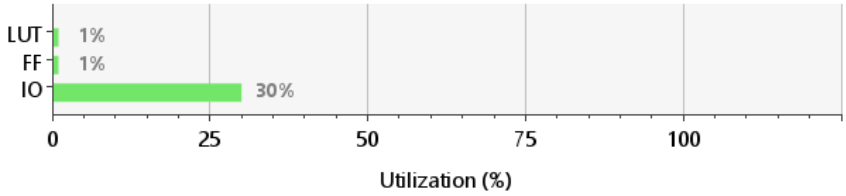
4. Caratterizzazione

Dopo aver validato la correttezza logica del progetto tramite simulazione behavioural, la fase successiva del flusso di progettazione riguarda la traduzione del codice VHDL in una configurazione fisica implementabile su FPGA. Questa sezione analizza i risultati ottenuti attraverso i processi di Sintesi e Implementazione. L'obiettivo di questa analisi non è più verificare se il circuito calcola correttamente, ma quantificare come lo fa in termini di prestazioni hardware. Andremo ad analizzare tre metriche fondamentali: Utilizzo di risorse, analisi temporale e consumo energetico. Questi ci vengono forniti dai reports di Vivado a seguito dell'implementazione.

4.1 Utilization Report

Fornisce una quantificazione dell'area di silicio occupata, con particolare attenzione al rapporto tra logica combinatoria (LUT) e celle di memoria (Flip-Flop).

Resource	Utilization	Available	Utilization %
LUT	59	53200	0.11
FF	105	106400	0.10
IO	60	200	30.00



Flip-Flop (FF)**:** Sono stati utilizzati 105 registri (0.10% del totale). Questo valore non è casuale, corrisponde infatti precisamente al risultato sommando i registri previsti in ogni stadio della pipeline:

1. **Input:** 6 vettori da 8 bit = 48 FF.
2. **Livello 1 $S1, S2, S3$:** 3 vettori da 9 bit = 27 FF.
3. **Livello 2 $S4, RS3$:** 1 vettore somma (10 bit) + 1 vettore ritardato (9 bit) = 19 FF.
4. **Output Stage R:** 1 vettore finale da 11 bit = 11 FF.

$$48 + 27 + 19 + 11 = 105\ FF$$

- **Look-Up Tables** (LUT)**:** Sono state utilizzate 59 LUT (0.11% del totale). Queste risorse implementano la logica combinatoria di calcolo (generazione dei segnali e catena dei riporti). Il rapporto tra Flip-Flop e LUT è di circa 1.8 : 1. Un rapporto superiore a 1 è caratteristico delle architetture a pipeline, dove si spende un'elevata quantità di area per la memorizzazione al fine di spezzare il percorso critico e alzare la frequenza.

- **IO:** Sono utilizzati 60 pin fisici (30% del totale disponibile). Anche questo dato è coerente con l'interfaccia definita nell'entity:
  - **Ingressi A, B, C, D, E, F**: $6 \times 8 = 48$ pin.
  - Uscita R: 11 pin.
  - Clock clk: 1 pin.

$$48 + 11 + 1 = 60\ pin$$

Il design risulta essere limitato dai pin di input/output piuttosto che dalla logica interna. L'occupazione delle risorse logiche (0.11% LUT) indica che l'architettura è estremamente compatta e potrebbe essere replicata centinaia di volte sullo stesso chip per elaborazioni parallele massive senza "soffocare" il dispositivo.

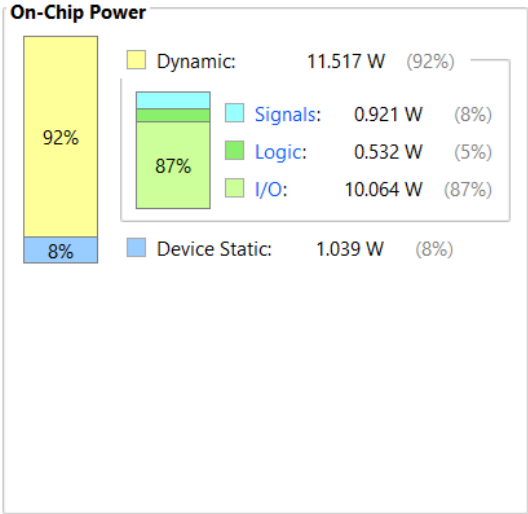
4.2 Power Report (analisi preliminare)

Fornisce una stima della dissipazione di potenza statica e dinamica

Power analysis from Implemented netlist. Activity derived from constraints files, simulation files or vectorless analysis.

Total On-Chip Power:	**12.557 W (Junction temp exceeded!)**
Design Power Budget:	**Not Specified**
Process:	typical
Power Budget Margin:	**N/A**
Junction Temperature:	**125,0°C**
Thermal Margin:	-84,8°C (-6,6 W)
Ambient Temperature:	25.0 °C
Effective ϑJA:	11,5°C/W
Power supplied to off-chip devices:	0 W
Confidence level:	Low

Launch Power Constraint Advisor to find and fix invalid switching activity



In una prima fase, l'analisi della dissipazione di potenza è stata condotta sulla netlist implementata in assenza di timing contraints. Il report evidenzia una stima di consumo totale pari a $12.557\ W$, che porterebbe la temperatura di giunzione oltre il limite critico di $125°$. Analizzando la ripartizione dei consumi, si nota che il 92% della potenza è dinamica e che l'87% di essa è assorbita dai blocchi di Input/Output.

4.3 Timing Constraints

Per ottenere una stima realistica e garantire il corretto funzionamento del circuito, definiamo dei vincoli temporali nel file xdc, creato tramite l'apposita sezione "Edit Timing Constraints".

$$create_{clock} - period\ 4.000\ - name\ perclk\ - waveform\ \{0.000\ 2.000\}[get_{ports} clk]$$

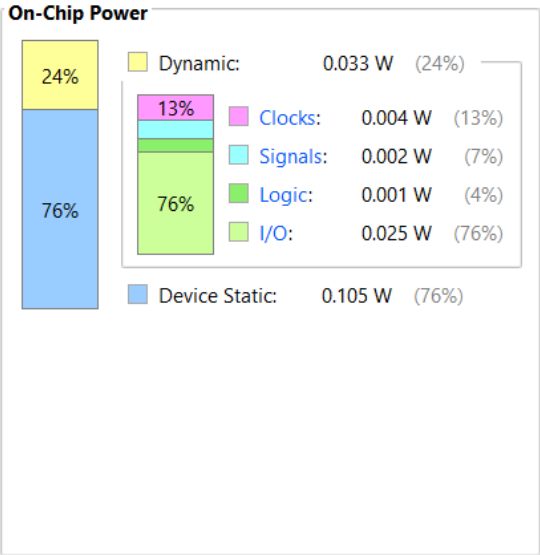
È stato impostato un clock con periodo $4\ ns$ corrispondente a una frequenza operativa di $250\ MHz$.

4.4 Analisi della Potenza a Regime (Constrained)

Power analysis from Implemented netlist. Activity derived from constraints files, simulation files or vectorless analysis.

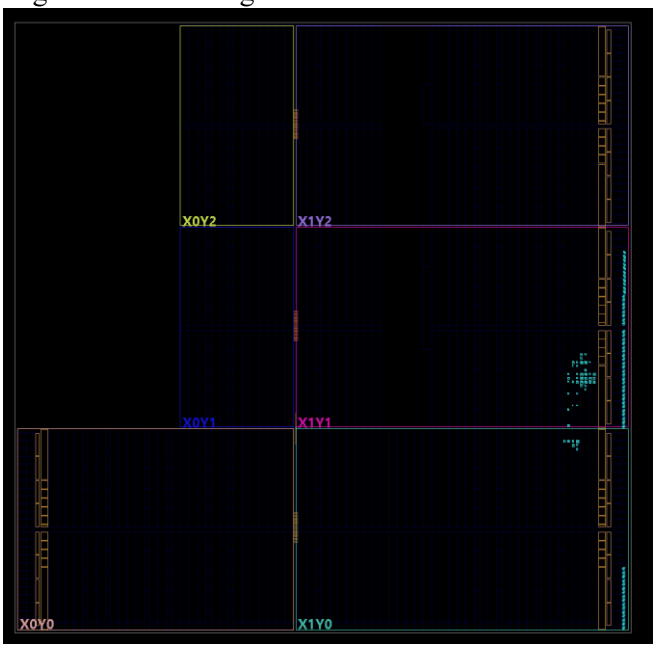
Total On-Chip Power:	**0.138 W**
Design Power Budget:	**Not Specified**
Process:	typical
Power Budget Margin:	**N/A**
Junction Temperature:	**26,6°C**
Thermal Margin:	58,4°C (4,9 W)
Ambient Temperature:	25.0 °C
Effective ϑJA:	11,5°C/W
Power supplied to off-chip devices:	0 W
Confidence level:	Low

Launch Power Constraint Advisor to find and fix invalid switching activity



Notiamo come dopo l'applicazione del vincolo temporale il power report mostra un drastico abbattimento dei consumi, delineando lo scenario operativo reale. Crolla da $12.557\ W$ a soli $0.138\ W$, e di conseguenza, la temperatura di giunzione simulata scende a $26.6°C$, un valore appena superiore alla temperatura ambientale predefinita ($25°\ C$). Questo elimina completamente il rischio termico evidenziato in precedenza, garantendo un ampio Thermal Margin di 58.4°C.

L'introduzione del clock ha inoltre invertito il profilo di consumo. La potenza statica si attesta a $0.105\ W$ e rappresenta ora la componente maggioritaria con il 76% del totale. La potenza dinamica, invece, è scesa a soli $0.033\ W$ visto che il consumo degli I/O, precedentemente stimato a 10 W, è ora ridimensionato a un valore realistico di $0.025\ W$.
Notiamo come la potenza dissipata dalla logica interna è irrisoria 0.001 W a dimostrazione dell'estrema efficienza dell'architettura pipelined che, nonostante l'alta frequenza (250 MHz), impegna pochissime risorse di commutazione.
Viene a conferma del basso consumo dinamico del circuito anche la Device View fornita in fase di implementazione, dove vengono evidenziate graficamente le risorse utilizzate dinamicamente:



4.5 Calcolo della Frequenza Massima

L'analisi del Timing Summary tiene conto dei ritardi fisici reali dovuti alla logica e al routing. Particolarmente di nostro interesse sono le righe dedicate al Design Timing Summary, specialmente attraverso il parametro WNS (Worst Negative Slack), rappresentante il peggior margine di tempo presente nel circuito.

```
------------------------------------------------------------------------------------------------
| Design Timing Summary
| ---------------------
------------------------------------------------------------------------------------------------

    WNS(ns)      TNS(ns)  TNS Failing Endpoints  TNS Total Endpoints      WHS(ns)      THS(ns)
    -------      -------  ---------------------  -------------------      -------      -------
      0.682        0.000                      0                   57        0.133        0.000


All user specified timing constraints are met.
```

Vediamo come nel nostro caso il WNS corrisponde a $+\,0{,}682\ ns$. Questo valore positivo indica che il circuito soddisfa pienamente i vincoli temporali imposti, non solo funziona a $250\ MHz$, ma dispone ancora di un margine di sicurezza di $0{,}682\ ns$ prima che il segnale arrivi troppo tardi per essere campionato.
Questo ci permette di calcolare il periodo minimo assoluto teorico e la frequenza massima (teorica stimata) che l'architettura sviluppata può sostenere su questa specifica FPGA come:

$$T_{min} = T_{clk} - WNS = 4{,}000\ ns - 0{,}682\ ns = 3{,}318\ ns$$

$$F_{max} = \frac{1}{T_{min}} = 301{,}38\ MHz$$

4.6 Analisi del Bottleneck

Dallo stesso report sulla temporizzazione possiamo trovare, analizzando la sezione Max Delay Paths, il percorso critico (ossia quello più lento) che determina il limite di velocità:

```
Max Delay Paths
--------------------------------------------------------------------------------------
Slack (MET) :             0.682ns  (required time - arrival time)
  Source:                 RegE/Q_reg[0]/C
                            (rising edge-triggered cell FDRE clocked by perclk  {rise@0.000ns fall@2.000ns period=4.000ns})
  Destination:            Sum3/S_reg[7]/D
                            (rising edge-triggered cell FDRE clocked by perclk  {rise@0.000ns fall@2.000ns period=4.000ns})
  Path Group:             perclk
  Path Type:              Setup (Max at Slow Process Corner)
  Requirement:            4.000ns  (perclk rise@4.000ns - perclk rise@0.000ns)
  Data Path Delay:        3.116ns  (logic 0.952ns (30.550%)  route 2.164ns (69.450%))
  Logic Levels:           4  (LUT3=1 LUT5=2 LUT6=1)
  Clock Path Skew:        -0.197ns (DCD - SCD + CPR)
    Destination Clock Delay (DCD):    4.585ns = ( 8.585 - 4.000 )
    Source Clock Delay      (SCD):    5.125ns
    Clock Pessimism Removal (CPR):    0.342ns
  Clock Uncertainty:      0.035ns  ((TSJ^2 + TIJ^2)^1/2 + DJ) / 2 + PE
    Total System Jitter     (TSJ):    0.071ns
    Total Input Jitter      (TIJ):    0.000ns
    Discrete Jitter          (DJ):    0.000ns
    Phase Error              (PE):    0.000ns
```

Osserviamo come questo trova riscontro nel percorso tra il registro di ingresso $RegE$ (Source) e il registro di somma $Sum3$ (Destination). Sono presenti 4 livelli logici composti dalle LUT in cascata per la logica di somma. Possiamo inoltre osservare la ripartizione del ritardo totale di $3{,}116\ ns$ in:

- **Logic:** $0{,}952\ ns = 30{,}5\%$
- **Route:** $2{,}164\ ns = 69{,}5\%$

Evidenzia come ritardo è dominato dal tempo di routing (percorrenza dei segnali attraverso i fili) piuttosto che dalla logica di calcolo.